

MERN STACK · WEEK 2 · HANDS-ON CSS

Dressing the Learning Tracker

Same HTML as last week. The only new line is one `<link>`. Everything else is CSS.

Instructor: Saddat Hasan · InfinitiBit

CSS = Cascading Style Sheets

- **Cascading** — a predictable set of rules decides which style wins when several target the same element
- **Style** — the visual presentation: colour, spacing, layout, motion
- **Sheets** — a document: a list of rules, kept separate from your HTML

A **stylesheet language** that describes how HTML elements are presented — on screen, on paper, or even in speech.

When rules collide, one wins

```
p { color: ... }
```

0-0-1

```
.lead { color: ... }
```

0-1-0

```
#intro { color: ... }
```

1-0-0 ✓ wins

Several rules can target the **same** element. The browser resolves the conflict in a fixed order:

- **Origin** — browser defaults < your stylesheet
- **Specificity** — id > class > type
- **Source order** — on a tie, the last rule wins

Structure and style, kept apart

- **HTML** = what things **are**
a heading, a list, a card, a form
- **CSS** = how they **look**
colours, sizes, spacing, position

Last week your tracker was raw HTML. This week the **markup has not changed** — we only added a few class names to hook onto. Everything you now see is CSS.

Three places styles can live

Inline

```
style="color:red"
```

Internal

```
<style> ... </style>
```

External

```
<link> + styles.css
```

All three can style the same page — and the cascade decides who wins.

Styles on the element itself

THE STYLE ATTRIBUTE

```
<p style="color: #4f46e5;  
        font-weight: 600;">  
  Styled right here.  
</p>
```

- A `style="..."` attribute on one element
- Styles **only** that element
- Sits at the top of the cascade — hard to override
- Fine for a quick test; **avoid** in real projects

A style block inside the page

IN THE <HEAD>

```
<head>
  <style>
    p { color: #4f46e5; }
  </style>
</head>
```

- A `<style>` block, usually in the head
- Styles **that one page**
- Good for a single page or a quick demo

A separate file — the default

```
<!-- index.html -->
<link rel="stylesheet" href="styles.css">
```

```
/* styles.css */
p { color: #4f46e5; }
```

- Styles live in a **.css** file, linked once
- One file styles **every** page
- The professional default — what your project uses

selector { property: value; }

The whole language is this one shape, repeated. Read it as: **“find these elements, and set these looks on them.”**

B

The grammar of CSS

Before any examples — the exact parts every rule is built from.

Every rule, taken apart



One rule = a selector + a declaration block. One declaration = property : value ;

The WHO — basic selectors

SELECTOR TYPES

```
h2          { }    /* every <h2>          */
.topic-card { }    /* class="topic-card" */
#week-1     { }    /* id="week-1"         */
a, button   { }    /* group: both         */
nav a       { }    /* <a> inside <nav>     */
```

- **tag** — by element name
- **.class** — the workhorse; reusable
- **#id** — one unique element
- **a, b** — group, same rules

The WHO — states & positions

PSEUDO-CLASSES & MORE

```
a:hover           { } /* pseudo-class */
input[type="text"] { } /* attribute   */
tr:nth-child(even) { } /* every 2nd row */
dialog::backdrop  { } /* pseudo-element */
```

- `:hover`, `:focus` — a state
- `[attr]` — matches an attribute
- `:nth-child()` — by position
- `::backdrop` — a generated part

A colon vs two colons

```
a:hover      { } /* pseudo-CLASS */  
p::first-line { } /* pseudo-ELEMENT */
```

- `:` pseudo-class — an element in a **state** or position (`:hover`, `:nth-child`)
- `::` pseudo-element — selects or **creates a part** of an element (`::before`, `::first-line`)

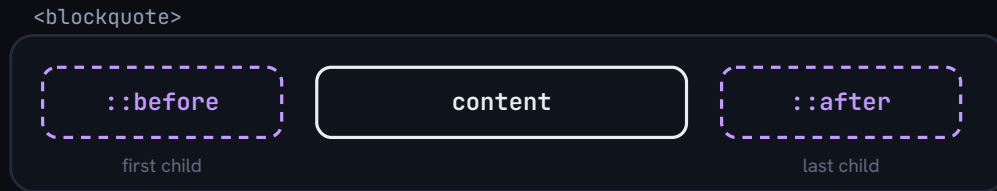
Legacy `:before` vs modern `::before`

SAME RESULT, TWO SPELLINGS

```
.tag:before { content: "#"; } /* CSS2 - still works */  
.tag::before { content: "#"; } /* CSS3 - use this */
```

- Both target the **same** pseudo-element and render identically
- The double colon `::` was added in CSS3 to tell pseudo-**elements** apart from pseudo-**classes**
- Write `::before` in new code; you'll still meet `:before` in older stylesheets

Insert content with CSS



- Create a generated child — `::before` first, `::after` last
- The `content` property is **required** — even `""`; without it, nothing renders
- Decorative only — **not** in the HTML source, so not for meaningful text
- Won't work on void tags (`img`, `input`, `br`)

Two real examples

DECORATIVE QUOTE MARKS

```
blockquote::before { content: "\201C"; }  
blockquote::after  { content: "\201D"; }
```

A MARKER IN YOUR TRACKER

```
.status--done::before { content: "\00BB "; }
```

- Curly quotes wrap a blockquote — **no extra HTML**
- A marker before every Done status, added purely in CSS
- Change the markup? Nothing — the decoration lives in the stylesheet

Four everyday, real uses

```
/* 1 · required-field asterisk */
label.required::after {
  content: " *";
  color: #dc2626;
}

/* 2 · breadcrumb separator */
.crumb + .crumb::before {
  content: " / ";
  color: #9aa3b4;
}
```

```
/* 3 · external-link arrow */
a[target="_blank"]::after {
  content: " ↗";
}

/* 4 · tooltip from an attribute */
[data-tip]:hover::after {
  content: attr(data-tip);
}
```

All four add UI **without touching the HTML** — a marker, a separator, an icon, and a tooltip, living entirely in CSS.

The WHAT and HOW

```
PROPERTY: VALUE;
```

```
.status {  
  display: inline-block;  
  padding: 0.15em 0.7em;  
  border-radius: 999px;  
  font-weight: 600;  
}
```

- A **property** is what you're changing
- A **value** is what you set it to
- End every one with a **semicolon** ;
- Stack as many as you like inside the braces

How a value is written

VALUES COME IN KINDS

```
color:      #4f46e5;      /* hex colour */
margin:     1.5rem;        /* length    */
width:      100%;          /* percentage */
gap:        var(--space-2);/* a variable */
background: rgba(0,0,0,.5);/* a function */
```

- **Colours** — hex, names, rgba()
- **Lengths** — px, rem, em, %
- **Functions** — var(), rgba(), minmax()

Notes the browser ignores

```
/* ... */  
  
/* This is a CSS comment.  
   Use it to label sections and  
   explain WHY, not just what. */  
  
body {  
  line-height: 1.6; /* comfy reading */  
}
```

Anything between `/*` and `*/` is ignored. Your `styles.css` uses them to number every section — read the file like a lesson.

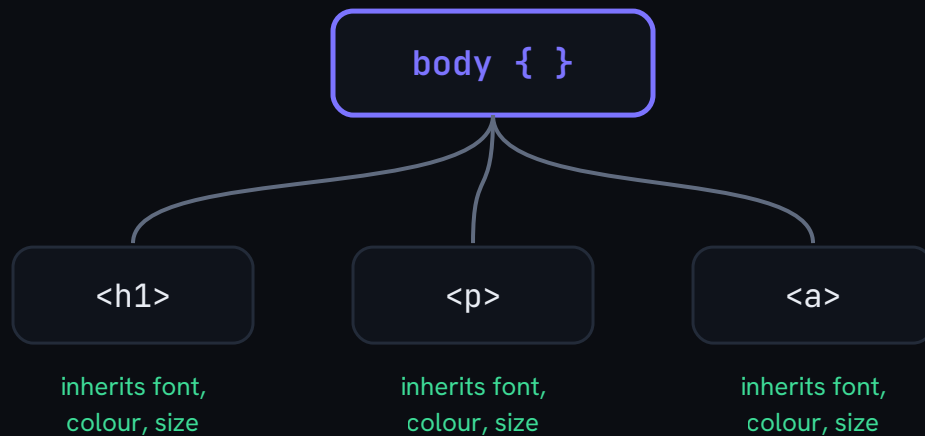
When rules collide

THE FIRST C IN CSS

```
a { color: grey; }  
.week-nav a { color: blue; } /* wins */  
  
/* two things decide the winner: */  
/* 1. specificity – how targeted */  
/* 2. source order – later beats */
```

More than one rule can hit the same element. The **more specific** selector wins; when specificity ties, the **later** rule wins.

Set once, flows down



set once on the parent → cascades down

FROM YOUR STYLES.CSS

```
body {  
  font-family: system-ui;  
  color: #1f2933;  
}
```

Text properties set on a parent are **inherited** by its children — so `body` sets the defaults for the whole page.

Every element is a box



YOUR RESET, SECTION 2

```
*, ::before, ::after {  
  box-sizing: border-box;  
}
```

Content, then **padding**, **border**, **margin**. border-box makes width mean the **whole** box — far more predictable.

C

Your stylesheet, for real

Every idea so far — now spotted in the file you're about to run.

Define once, use everywhere

CUSTOM PROPERTIES

```
:root {  
  --color-accent: #4f46e5;  
  --space-3: 1rem;  
}  
  
a { color: var(--color-accent); }  
button { background: var(--color-accent); }
```

:root is the whole document. Names starting with **--** are your own reusable values, read with **var()**.

Change the colour in **one** place, the whole page updates.

Base + modifier: WHAT vs HOW

FROM STYLES.CSS

```
.status      { /* shared pill shape */ }  
.status--done { background: #dcfce7; }  
.status--learning { background: #fef3c7; }
```

FROM INDEX.HTML

```
<span class="status status--done">Done</span>
```

The HTML says **what** the status is (a class name). The CSS decides how it **looks**. One shape, three colour swaps.

Lay children out in a row

`display: flex` puts children in a row and shares space

`gap` lets them

→ main axis (row)

Week 1

gap

Week 2

gap

Week 3

THE WEEK NAV

```
.week-nav ul {  
  display: flex;  
  flex-wrap: wrap;  
  gap: var(--space-2);  
}
```

A gallery that reflows itself



THE TOPIC-CARD GRID

```
section {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fill, minmax(260px, 1fr));  
}
```

State & position, live

TABLE + FORM

```
tbody tr:nth-child(even) {  
  background: #fafbfc; /* zebra */  
}  
tbody tr:hover {  
  background: #eef2ff; /* row under cursor */  
}  
input:focus-visible {  
  outline: 2px solid var(--color-accent);  
}
```

- **:nth-child(even)** stripes the table with no extra classes
- **:hover** reacts to the cursor
- **:focus-visible** shows a ring for keyboard users

Don't snap — animate

THE CARD LIFT

```
.topic-card {  
  transition: transform .15s ease,  
             box-shadow .15s ease;  
}  
  
.topic-card:hover {  
  transform: translateY(-3px);  
}
```

A **transition** tells the browser: when this property changes, ease it over time instead of jumping. The cards lift gently on hover — a tactile, dashboard feel.

Different rules on small screens

RESPONSIVE POLISH

```
@media (max-width: 600px) {  
  main, section {  
    grid-template-columns: 1fr; /* one per row */  
  }  
  h1 { font-size: 1.6rem; }  
}
```

A `@media` block applies its rules **only** when the condition is true — here, narrow phones. The cards collapse to a single column.

Same markup. New skin.

Your `index.html` barely changed — a few class names, one `<link>`. Every card, pill, and layout you just saw came entirely from `styles.css`. That separation is the whole point.

Rules that keep CSS sane

- Keep **structure in HTML, style in CSS** — prefer an external stylesheet
- Prefer **classes over ids**; keep specificity low and flat
- Avoid **!important** — a last resort, not a tool
- Name consistently — kebab-case, BEM-style `block__el--modifier` (your `.status--done`)
- Reuse with **variables**; don't repeat values
- Let **flex & grid** do layout; go mobile-first

Same goal, two ways

AVOID

```
/* over-specific + !important */  
#main .card p.title {  
  color: red !important;  
}
```

PREFER

```
/* one flat, reusable class */  
.card-title {  
  color: var(--color-accent);  
}
```

The second is easier to reuse, easier to override, and reads at a glance.

Low specificity + good names = calm CSS.

The grammar in one place

THE SHAPE OF EVERYTHING

```
selector {           /* WHO */
  property: value;    /* WHAT: HOW */
}                    /* ; ends each line */
```

- **Selector** — which elements
- **Property** — what to change
- **Value** — what to set it to
- **Cascade + inheritance** — how conflicts and defaults resolve

NEXT WEEK

HTML gave it structure. CSS gave it **style**.

Next: JavaScript makes it **move** — the form finally works, and the progress count computes itself.
Commit your work and tag v2-css.